

人工智能技术实验

Artificial Intelligence Technology Experiment

许丽

南开大学人工智能学院

xuli@nankai.edu.cn





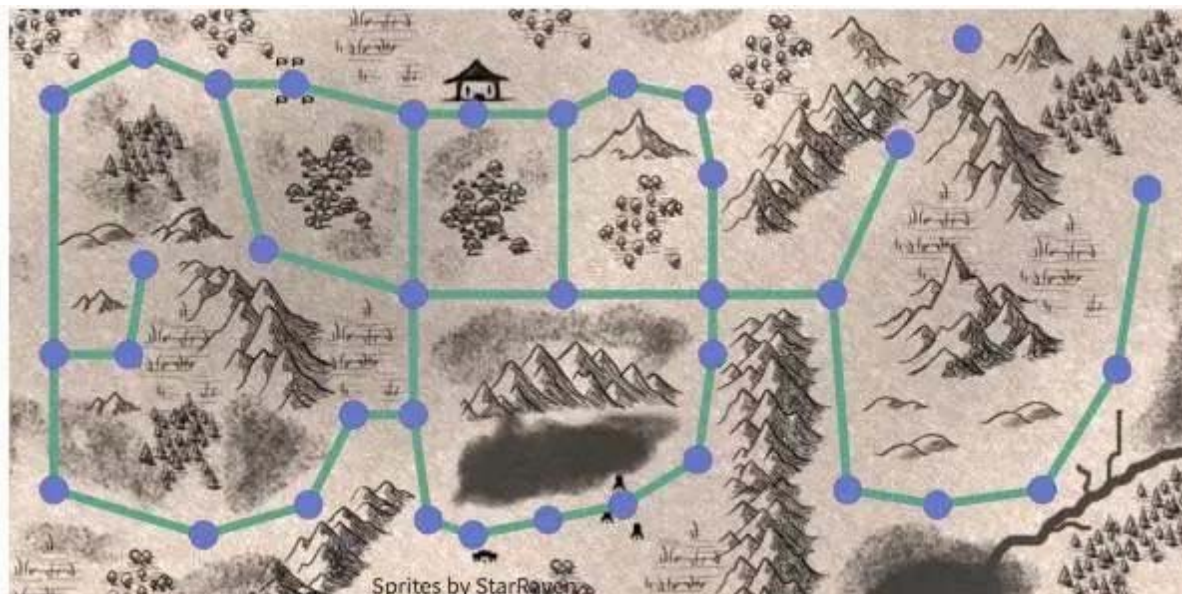
L3 A*算法实现 (问题求解)

寻路问题

A*算法



- 游戏中为了模仿人物行走的真实体验，他们会选择最近路线达到目的地，期间会避开高山或者湖水，绕过箱子或者树林，直到走到你所选定的目的地。
- 静态路网中**最短时间**内找到**最短路线**的问题。

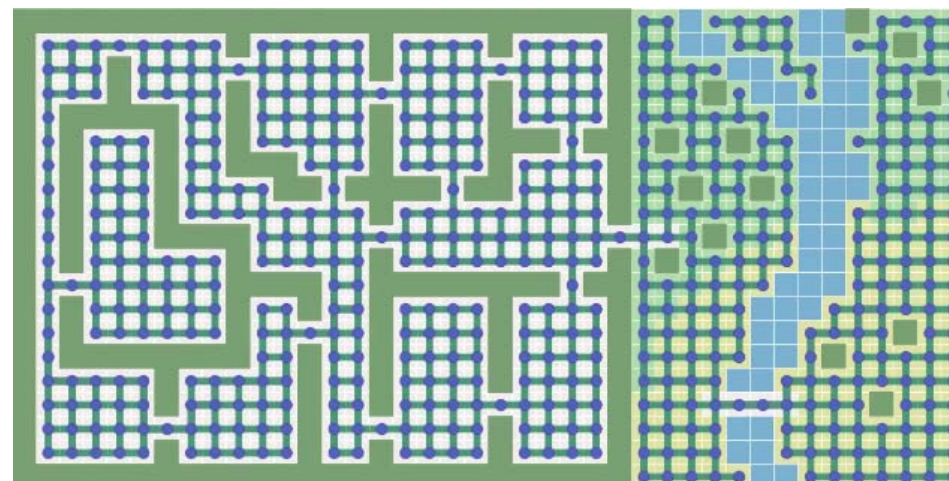
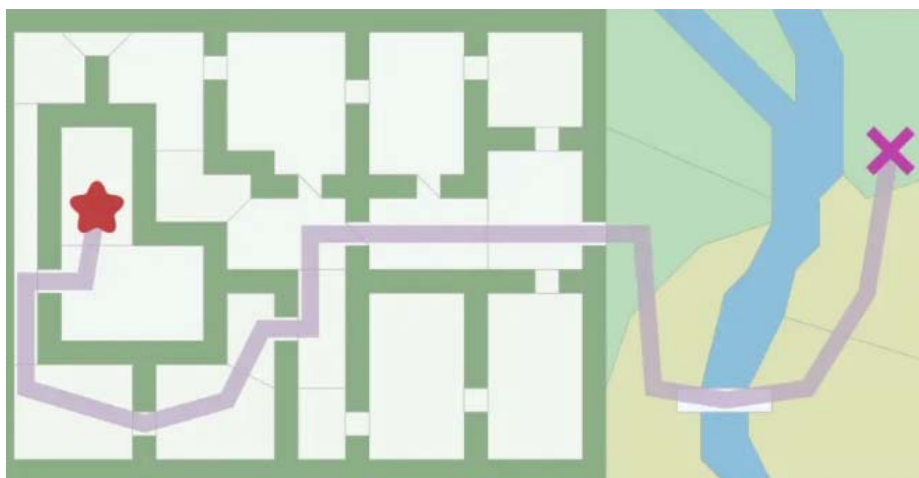


寻路问题

A*算法



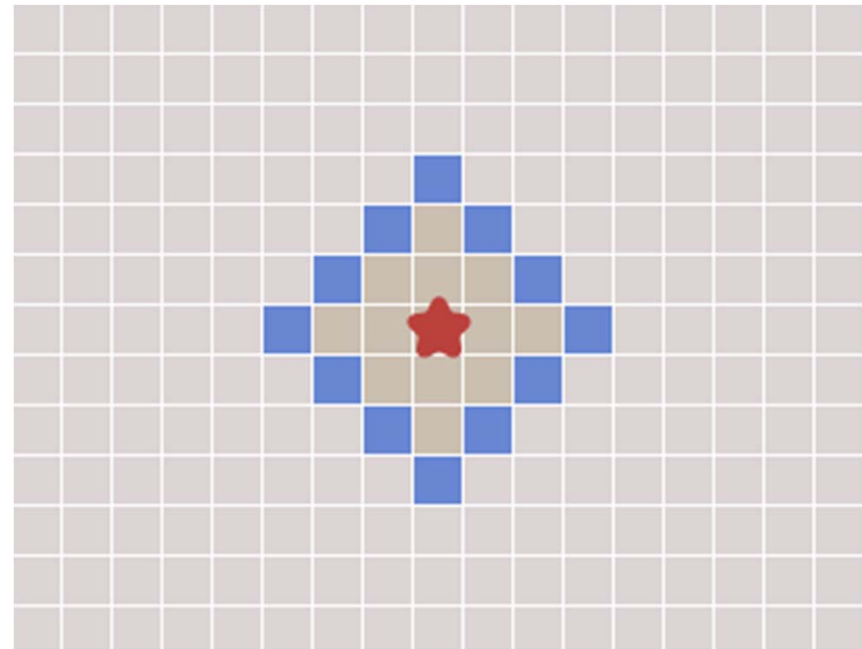
- 抽象成数学问题，常见的是把地图切成网格，节点和边。
- 减小搜索范围，控制搜索规模，降低实际问题的复杂度。
- 无信息搜索、有信息搜索。





无信息搜索（盲目搜索）

- 深度优先(DFS): 不适合于最小路径;
- 广度优先(BFS): 效率低下, 没有权值;



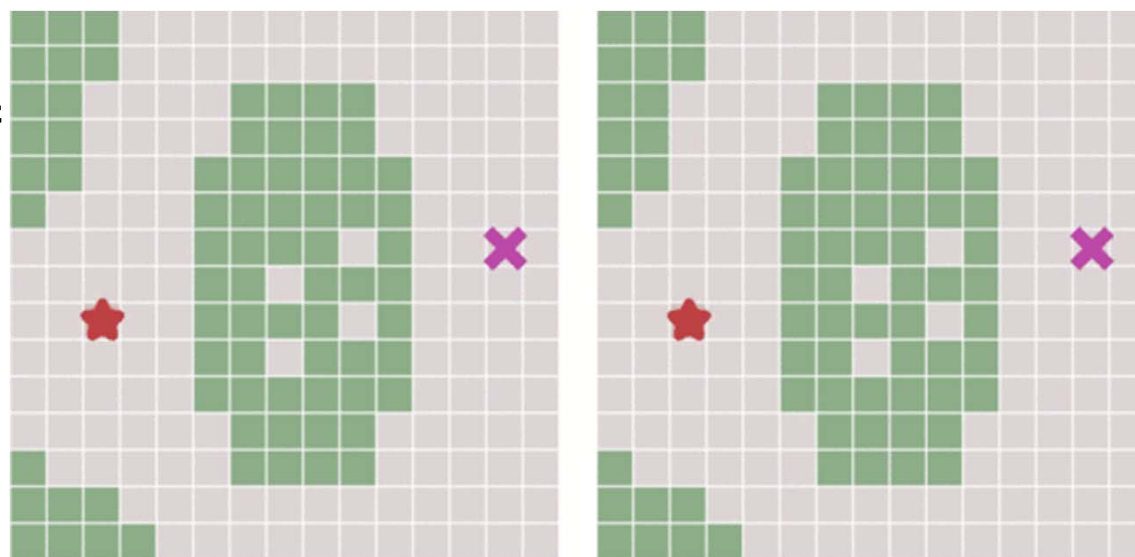
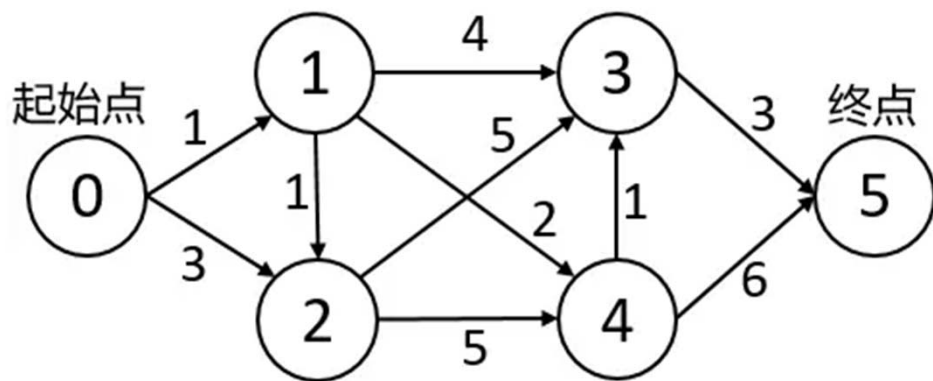
搜索算法

A*算法



无信息搜索（盲目搜索）

- Dijkstra算法：加入权值（成本），优先队列结构（所有待遍历的点），不具有方向性。



搜索算法

A*算法



有信息搜索（启发式搜索）

- 贪心算法（到目标顶点的距离进行排序，不是最短路径）
- A*算法（Dijkstra+贪心算法，不同的估价函数影响性能）

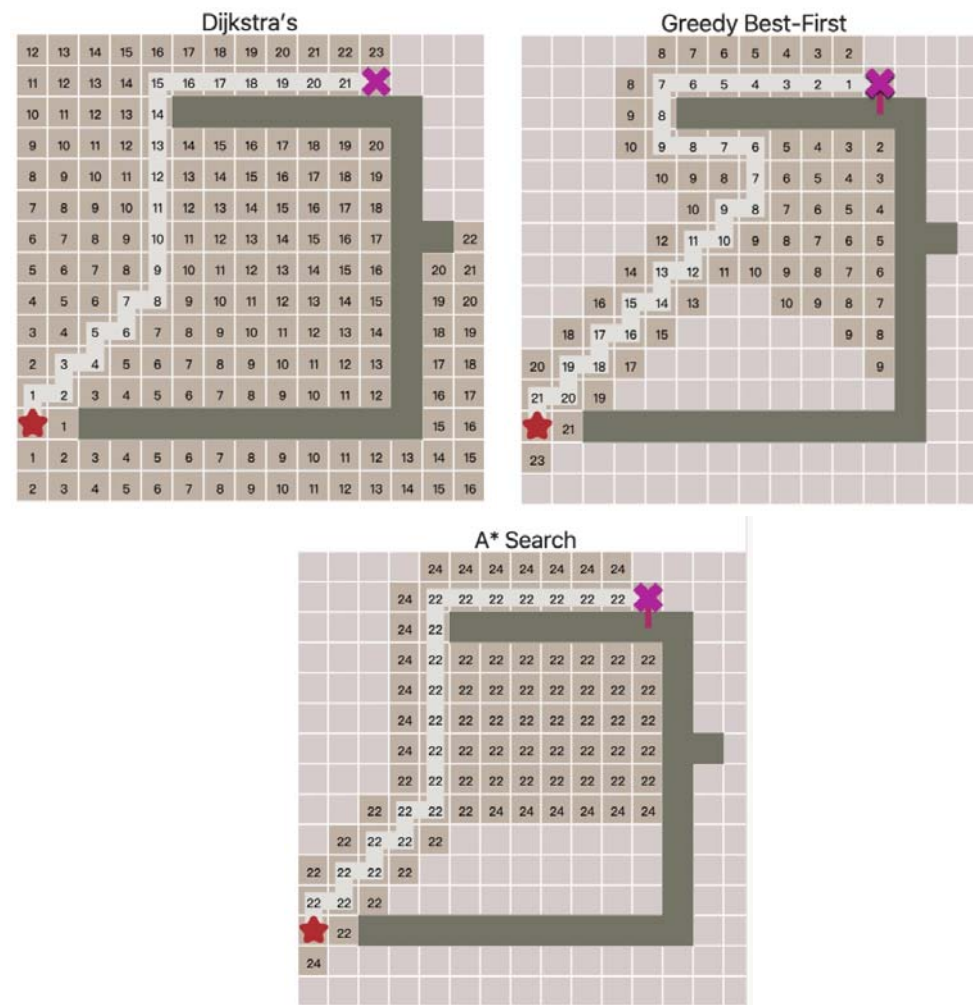
算法核心：

$$f'(n) = g'(n) + h'(n)$$

$g'(n)$ 是节点 n 距离起点的代价，

$h'(n)$ 是节点 n 距离终点的预计代价；

- 比较：计算量不同





- 估价：**忽略路径中的障碍物，不考虑可通行性**
- $h(n) = 0$ ，该A*算法已经退化为了Dijkstra算法，一定能解得最优解但是运行效率最低。
- $h(n) = h^*(n)$ ，该A*算法不仅能够保证一定能解得最优解，而且运行效率在所有能保证解得最优解的A*算法中是最高的。其中 $h^*(n)$ 表示结点 n 到终点的实际代价。
- $h(n)$ 对所有的 n 有 $h(n) \leq h^*(n)$ ，则该A*算法能保证一定能得到最优解，但是效率略低于上述 $h(n) = h^*(n)$ 的情况。
- $h(n)$ 存在 n 使得 $h(n) > h^*(n)$ ，则该A*算法不一定能得到最优解。



- **曼哈顿距离**，从当前格到目的格之间水平和垂直的方格的数量总和

$$h(s) = |s.x - goal.x| + |s.y - goal.y|$$

- **切比雪夫距离**，各坐标数值差绝对值的最大值

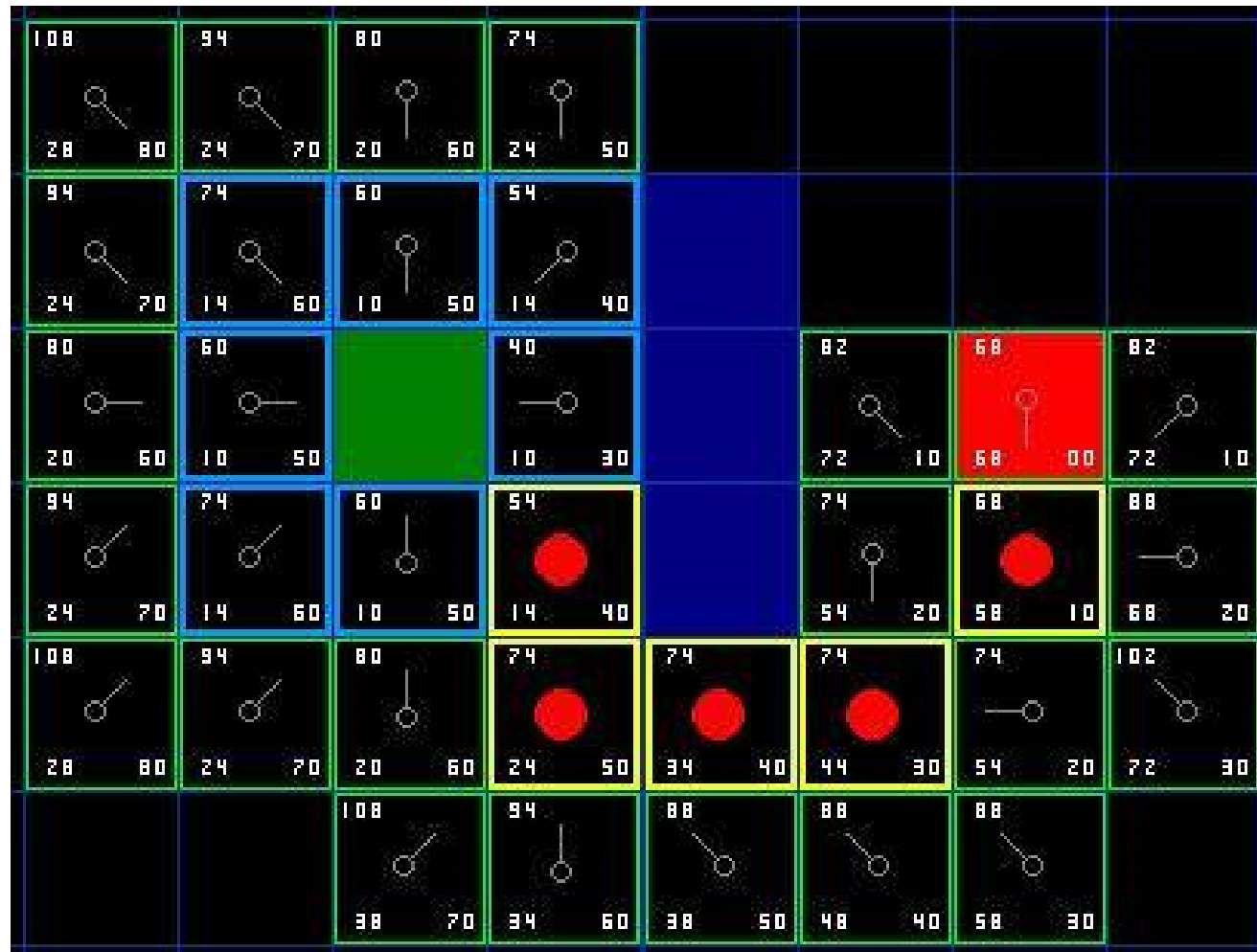
$$h(s) = \max(|s.x - goal.x|, |s.y - goal.y|)$$

- **欧氏距离**，两个点之间的真实距离（任意方向移动）

$$c = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

实验内容展示

A*算法



1. 简化搜索区域（地图栅格化）

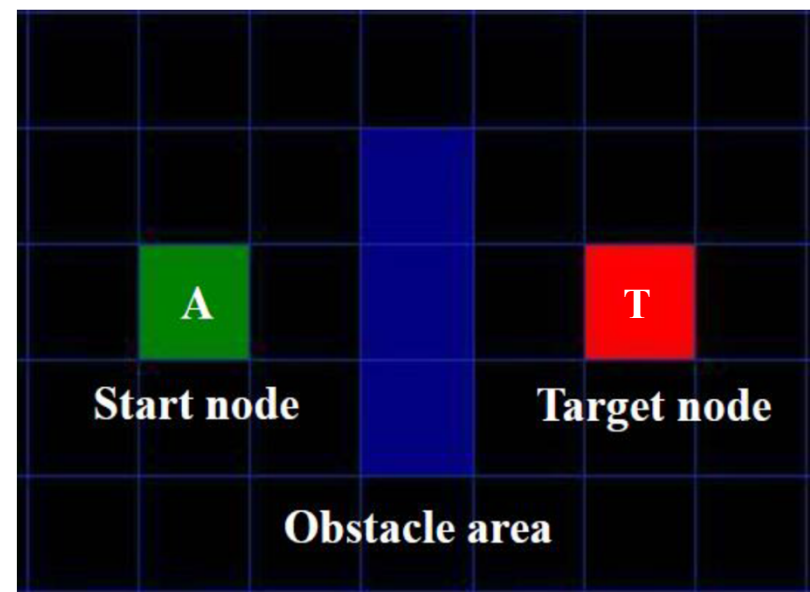
A*算法



- 把搜索区域划分成方形网格;
- 用二维数组保存方形网格;
- 数组的每一个元素是网格的一个方块, 方块被标记为**可通过**的和**不可通过**的;
- 路径描述为从A到T经过的方块的集合
- 创建两个状态集合:

open list 存储待考察的节点

close list 存储已经考察完毕的节点



绿色的是起点A
红色是终点T
蓝色是中间的墙。

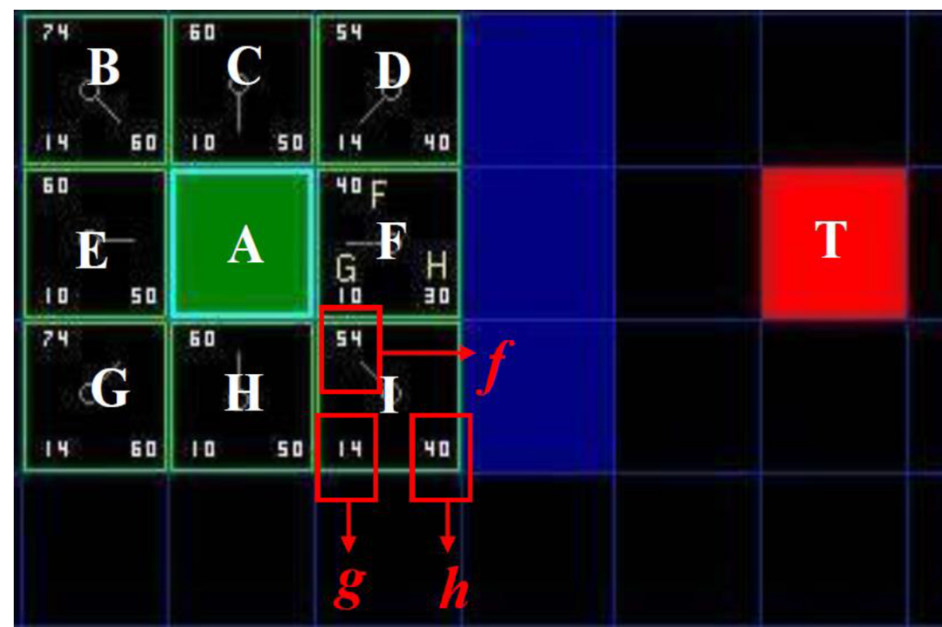
2. 估价函数定义

A*算法



$$f'(n) = g'(n) + h'(n)$$

- $g'(n)$ 是节点 n 距离起点的代价, $h'(n)$ 是节点 n 距离终点的预计代价;
- G 值对于网格形式的图, 水平或者垂直移动的耗费为10, 对角线方向耗费为14;
- H 值使用曼哈顿方法;
- 左上角为 F , 左下角为 G , 右下角为 H 。

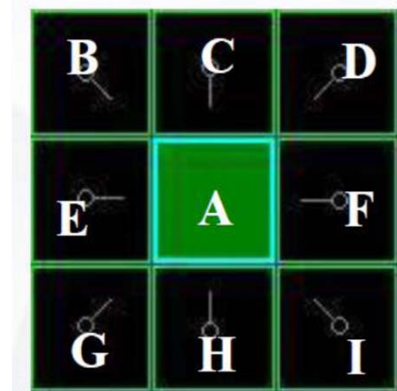
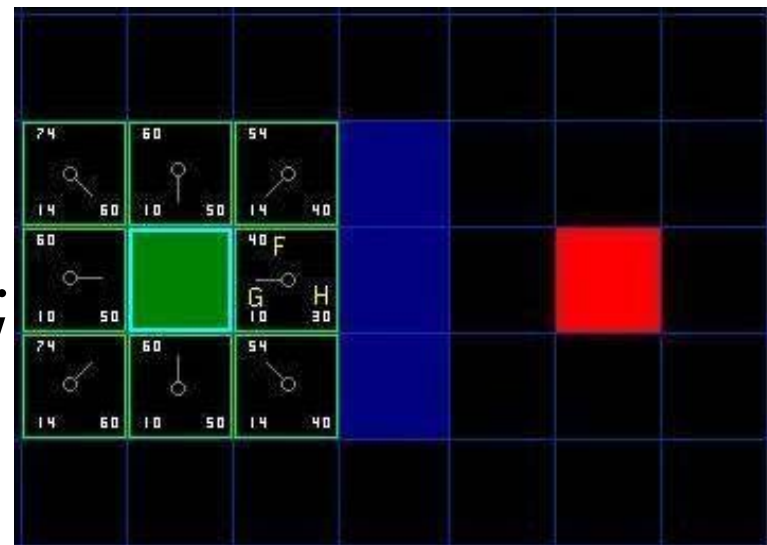


3.开始搜索 (openlist、closelist)

A*算法



- 从点A开始，把它作为待处理点存入一个openlist: **openlist{A} closelist{}**
- 从openlist中删除点A，把它加入到closelist;
- 寻找起点周围所有可到达或者可通过的方格，跳过有墙或其他无法通过地形的方格。也把他们加入openlist。为所有这些方格保存点A作为“父方格”。



openlist{B C D E F G H I} closelist{A}

4.搜索过程

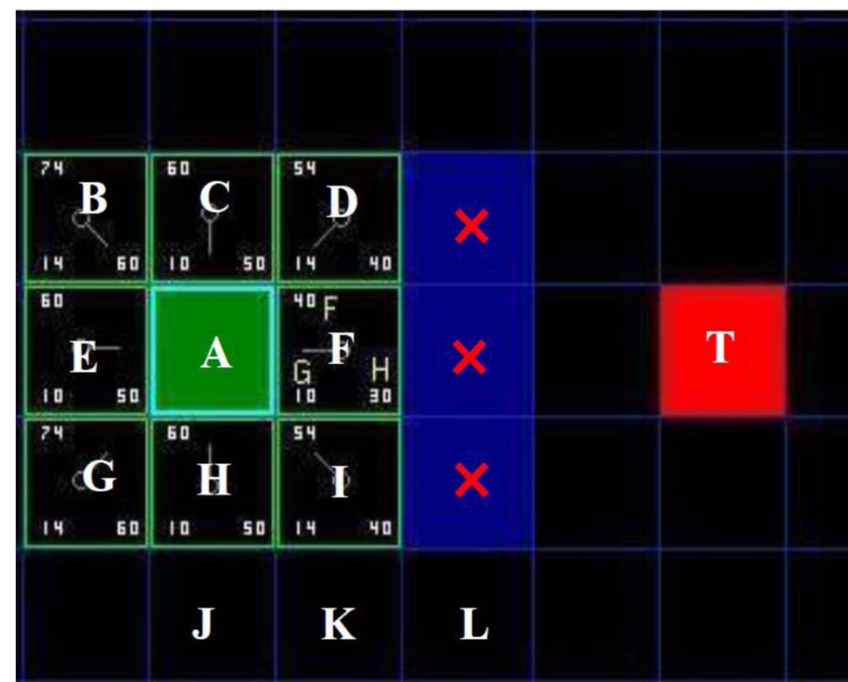
A*算法



- 从openlist选择F值最低的方格，把它从openlist删除，然后添加到closelist。
- 检查该方格所有相邻格子。跳过已经在closelist中的或者不可通过的，添加到openlist。
- 如果某个相邻格已经在openlist里了，检查现在的这条路径是否更好(G值作参考)，更好则更换，并将其父节点改为当前选中的节点。

openlist{B C D E G H I} closelist{A F}

openlist{B C D E G H J K } closelist{A F I}

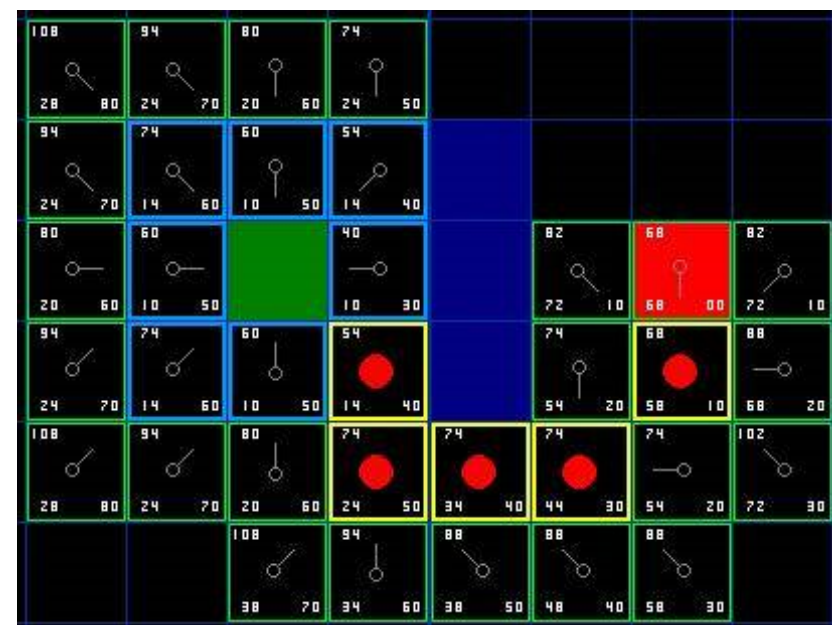


4.搜索过程

A*算法

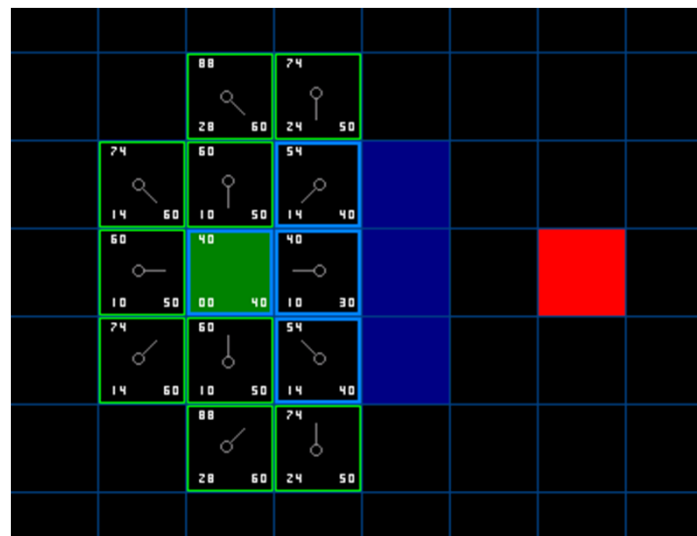
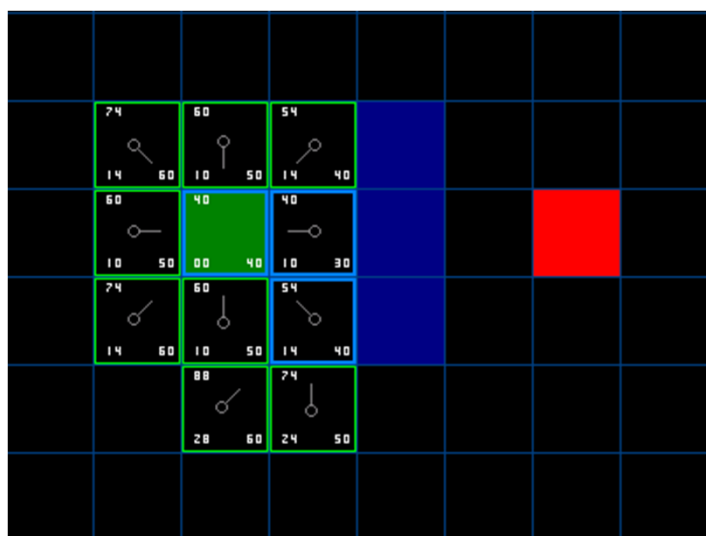
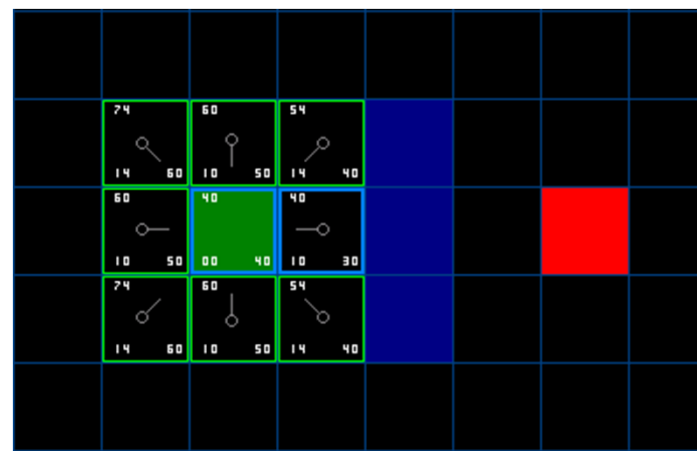


- 路径是通过反复遍历openlist并且选择具有最低F值的方格来生产的
- 停止：
 - 路径被找到：当把目标格添加进了openlist；
 - 路径不存在：没有找到目标格， openlist已空。
- 保存路径。从目标格开始，沿着每一格的父节点移动直到回到起始格。



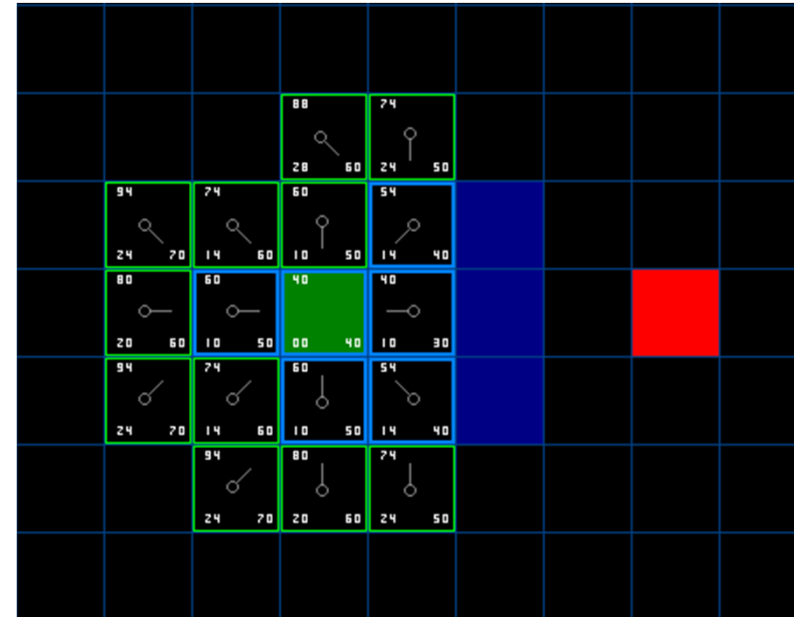
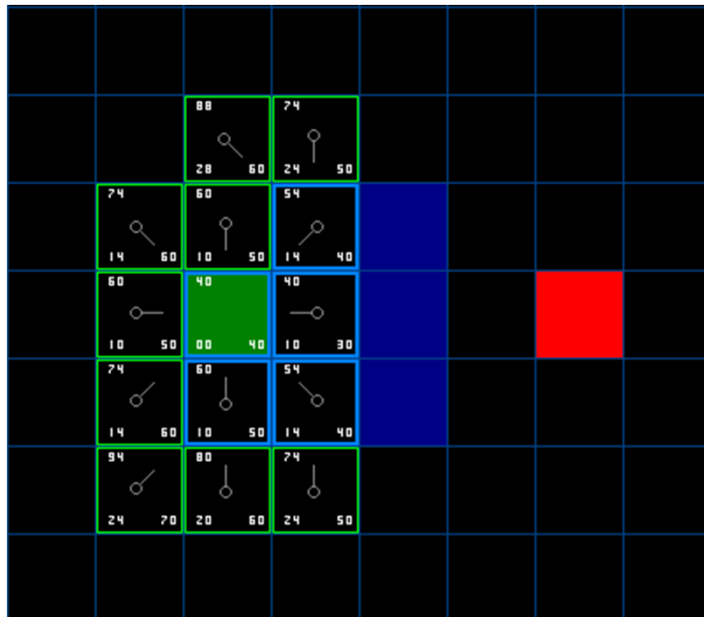
4.搜索过程

A*算法



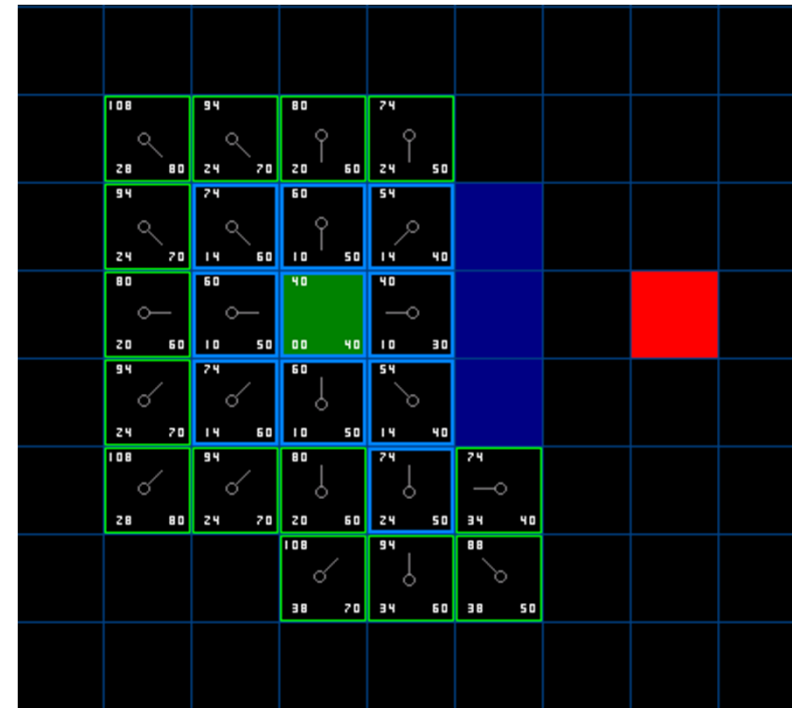
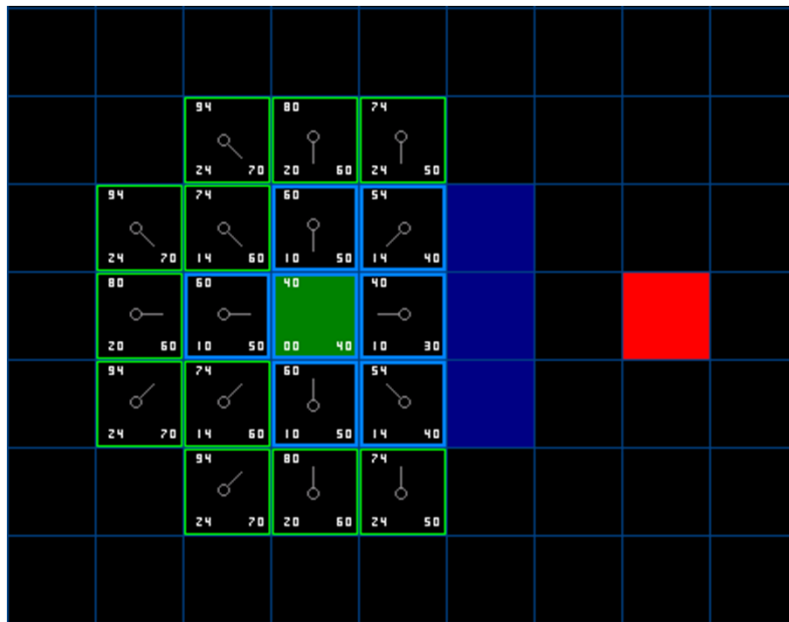
4.搜索过程

A*算法



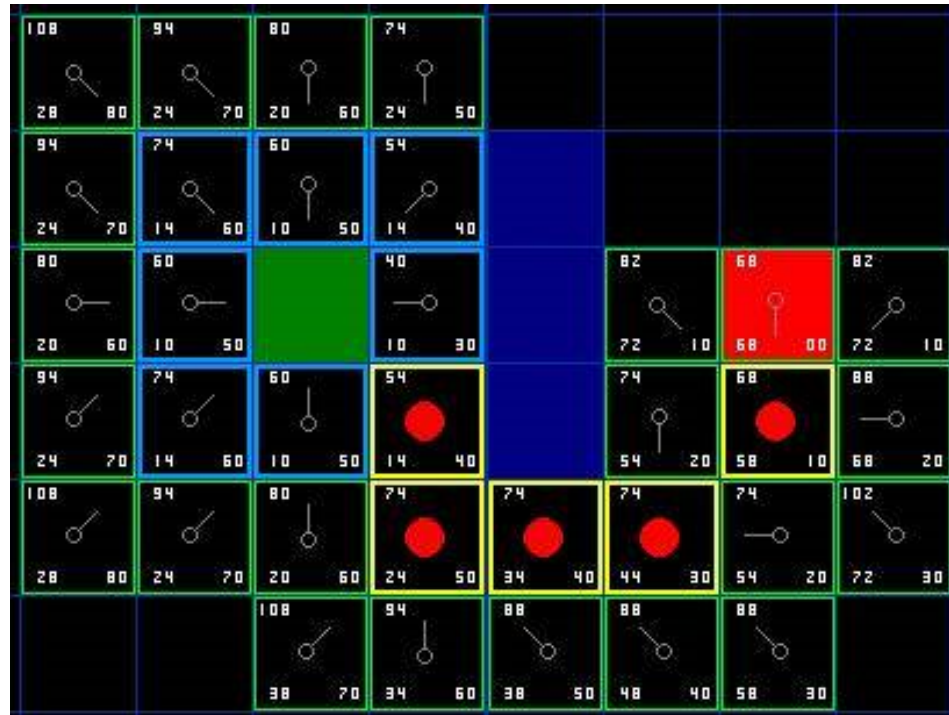
4.搜索过程

A*算法



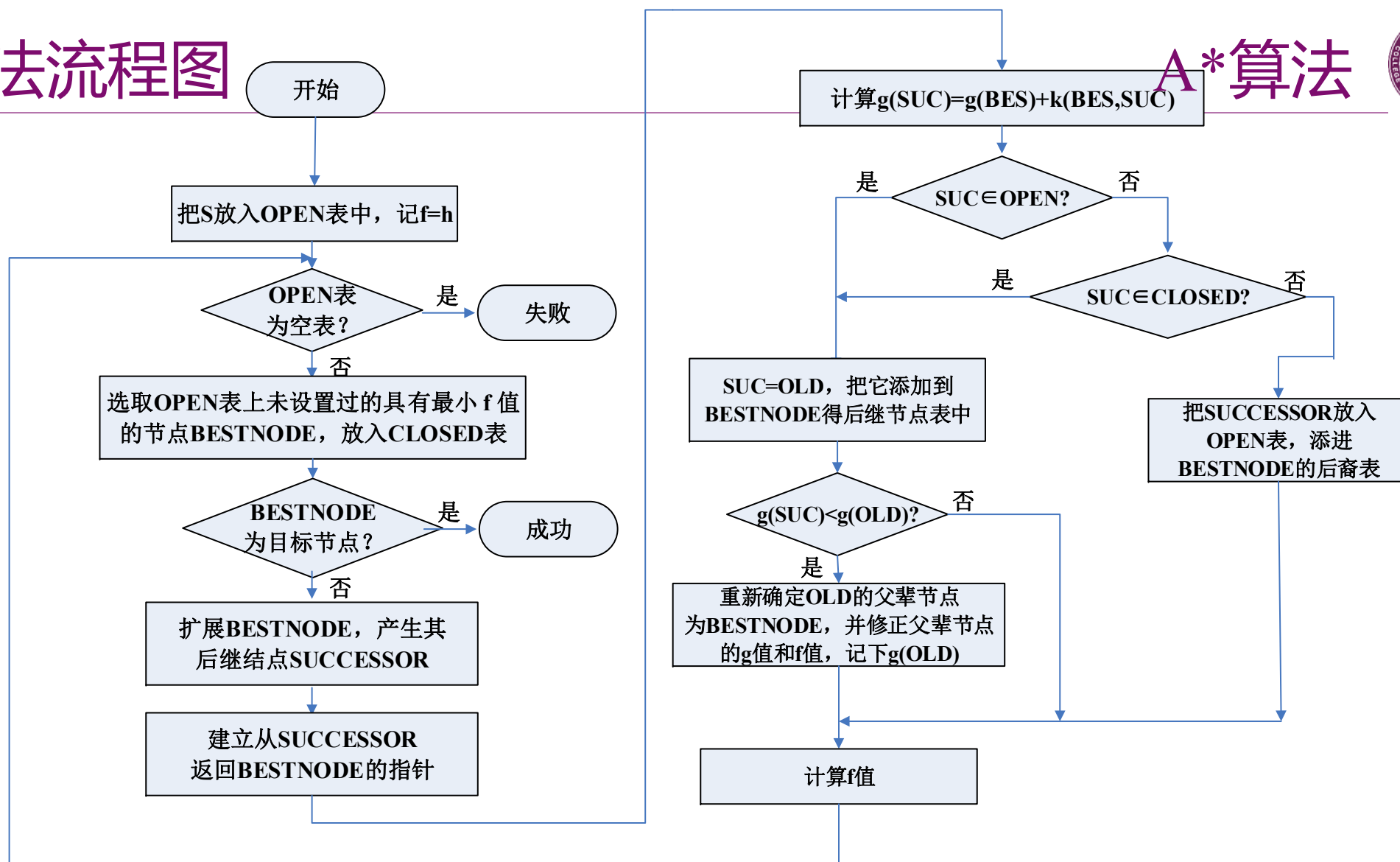
5.搜索结果

A*算法



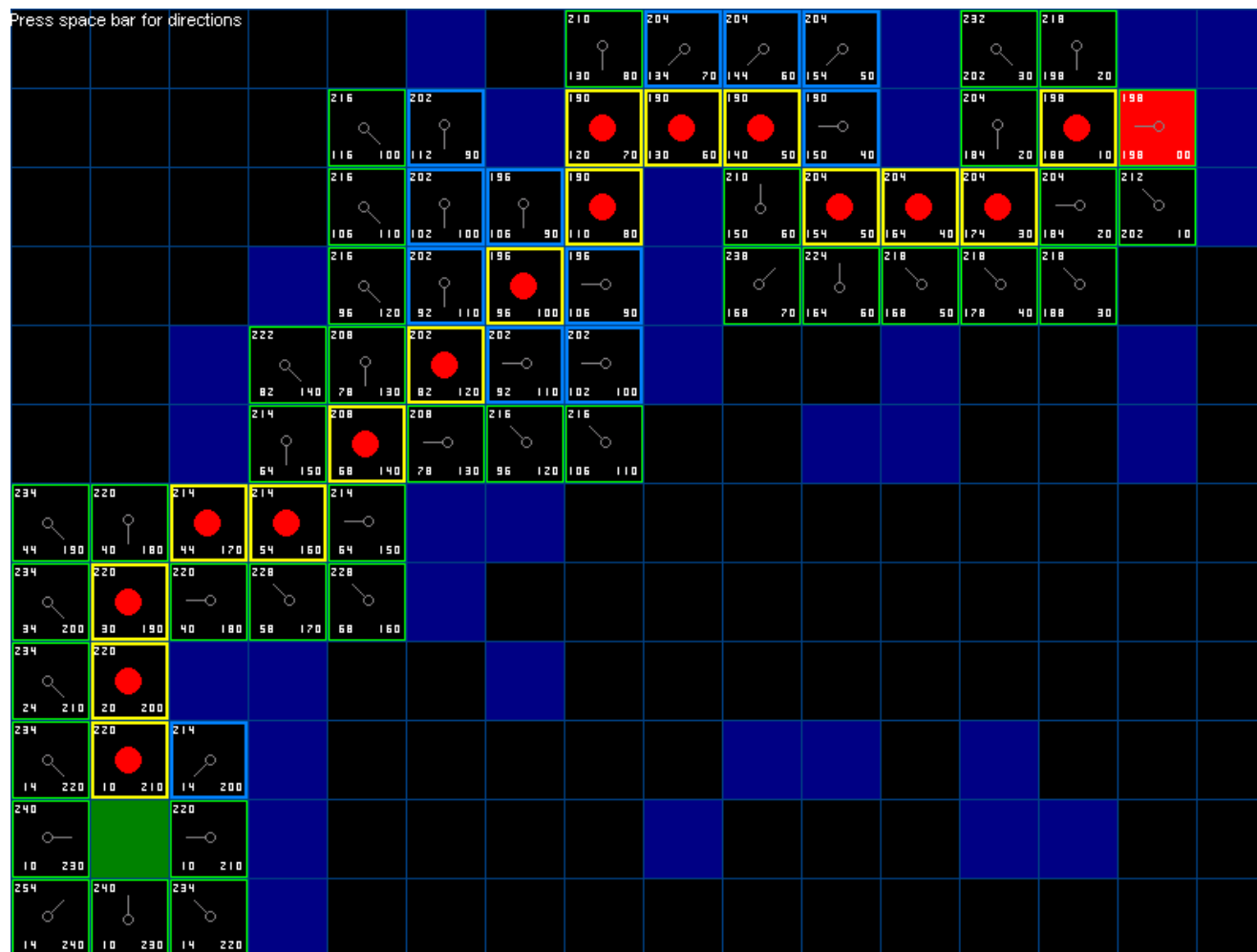
算法流程图

A*算法



网格地图寻路

A*算法



注意事项

A*算法



- openlist: 每次找出代价最小的方格
 - 不排序: 遍历
 - 排序: 排序数组 排序链表 优先队列等
- 移动方向
 - 四个方向
 - 八个方向
- H的不同定义
- 墙角: 是否可以穿越





- 地图绘制，初始化
- 鼠标事件响应：起点，终点，墙的设置
- 键盘事件响应：单步运行，运行暂停，重置等功能



初始化open_list和close_list;

- * 将起点加入open_list中, 并设其移动代价为0;

- * 如果open_list不为空, 则从open_list中选取移动代价最小的节点n:

 - * 如果节点n为终点, 则:

 - * 从终点开始逐步追踪parent节点, 一直达到起点;

 - * 返回找到的结果路径, 算法结束;

 - * 如果节点n不是终点, 则:

 - * 将节点n从open_list中移除, 并加入close_list中;

 - * 遍历节点n所有的邻近节点:

 - * 如果邻近节点m在close_list中, 则:

 - * 跳过, 选取下一个邻近节点

 - * 如果邻近节点m在open_list中, 则:

 - * 计算起点经n到m的g值

 - * 如果此时计算出来的g值小于原来起点经m的原父节点到m的g值:

 - * 更新m的父节点为n, 重新计算m的移动代价 $f=g+h$

 - * 否则, 跳过

 - * 如果邻近节点m不在open_list也不在close_list中, 则:

 - * 设置节点m的parent为节点n

 - * 计算节点m的移动代价 $f=g+h$

 - * 将节点m加入open_list中

A*算法实验要求

- 基本要求：实现无信息搜索算法（Dijkstra）、有信息搜索算法（A*算法）（90分要求）
- 选做内容：可视化界面（100分要求）
- 提交时间：三周
- 提交方式：在线提交（*实验报告、*源代码、demo等）
- 迟交问题：扣分
- 抄袭问题：不及格处理！

多项选择题

题目：关于A*算法，以下说法正确的是

- A. A*算法是一种启发式搜索算法
- B. 估价函数会影响A*算法的行为
- C. 估计函数中的 $g'(n)$ 为节点 n 距离起点的实际代价
- D. 估计函数中的 $h'(n)$ 为节点 n 距离终点的实际代价

S (C
VJ C P M U!

